Notebook setup

Run the next cell first. In Google Colab it clones the course repository, then imports the libraries. If you cloned the repository and run the notebook locally (see the README), it only does the imports.

Hybrid models

A simple model for cell expansion, accounting for the growth of the viable cell density (VCD) and the consumption of glucose (Glc), is used to show how hybrid models are trained and how they predict new experimental conditions.

A hybrid model combines mechanistic knowledge (the mass balances, written as ordinary differential equations) with data-driven components (here Gaussian processes) that learn the unknown kinetic rates from data.

Below we first define the "true" process (the in silico model that generates the data) and then the two hybrid models used in this notebook. Each consists of two functions: the first defines the ODE system that describes the dynamics, the second solves the equations for given parameters and initial conditions.

```
def ode_fcn(t, y, feed):
    # true process: Monod growth kinetics and glucose consumption
    VCD = y[0]
    Glc = y[1]
    # growth rate
    mu = Glc/(5+Glc)
    # mass balances
    dVCD_dt = mu*VCD
    dGlc_dt = -(0.5*mu+0.05*Glc)*VCD+feed
    dy = [dVCD_dt, dGlc_dt]
    return dy

def run_experiment(VCD_0, Glc_0, feed, t_end, time_step=0.25):
    fun = lambda t, y: ode_fcn(t,y,feed)
    y0 = [VCD_0, Glc_0]
    t_span = np.arange(0, 0.5*round(2*t_end)+time_step, time_step)
    sol = solve_ivp(fun, [t_span[0], t_span[-1]], y0, method='LSODA', t_eval=t_span,
rtol=1e-6, atol=1e-6)
    t = sol.t.tolist()
    y = sol.y.T
    VCD = y[:, 0]
    Glc = y[:, 1]
    return t, VCD, Glc
```

```
def ode_fcn_1st_hybrid(t, y, feed, g_mld, k_mld):
    # mass balances with the total growth and consumption rates learned by the GP
models
    state = y.reshape(1, -1)          # (VCD, Glc) as one sample for the GP
    dVCD_dt = g_mld.predict(state)[0]
    dGlc_dt = -k_mld.predict(state)[0] + feed
    return [dVCD_dt, dGlc_dt]

def run_1st_hybrid(VCD_0, Glc_0, feed, t_end, g_mld, k_mld, time_step=0.25):
    fun = lambda t, y: ode_fcn_1st_hybrid(t, y, feed, g_mld, k_mld)
    y0 = np.array([VCD_0, Glc_0])
    t_span = np.arange(0, 0.5 * round(2 * t_end) + time_step, time_step)
    sol = solve_ivp(fun, [t_span[0], t_span[-1]], y0, method='LSODA', t_eval=t_span,
rtol=1e-6, atol=1e-6)
    t = sol.t.tolist()
    y = sol.y.T
    VCD = y[:, 0]
    Glc = y[:, 1]
    return t, VCD, Glc
```

```
def ode_fcn_2nd_hybrid(t, y, feed, g_mld, k_mld):
    # mass balances with the cell-specific rates learned by the GP models, multiplied
by VCD
    state = y.reshape(1, -1)          # (VCD, Glc) as one sample for the GP
    dVCD_dt = g_mld.predict(state)[0] * y[0]
    dGlc_dt = -k_mld.predict(state)[0] * y[0] + feed
    return [dVCD_dt, dGlc_dt]

def run_2nd_hybrid(VCD_0, Glc_0, feed, t_end, g_mld, k_mld, time_step=0.25):
    fun = lambda t, y: ode_fcn_2nd_hybrid(t, y, feed, g_mld, k_mld)
    y0 = np.array([VCD_0, Glc_0])
    t_span = np.arange(0, 0.5 * round(2 * t_end) + time_step, time_step)
    sol = solve_ivp(fun, [t_span[0], t_span[-1]], y0, method='LSODA', t_eval=t_span,
rtol=1e-6, atol=1e-6)
    t = sol.t.tolist()
    y = sol.y.T
    VCD = y[:, 0]
    Glc = y[:, 1]
    return t, VCD, Glc
```

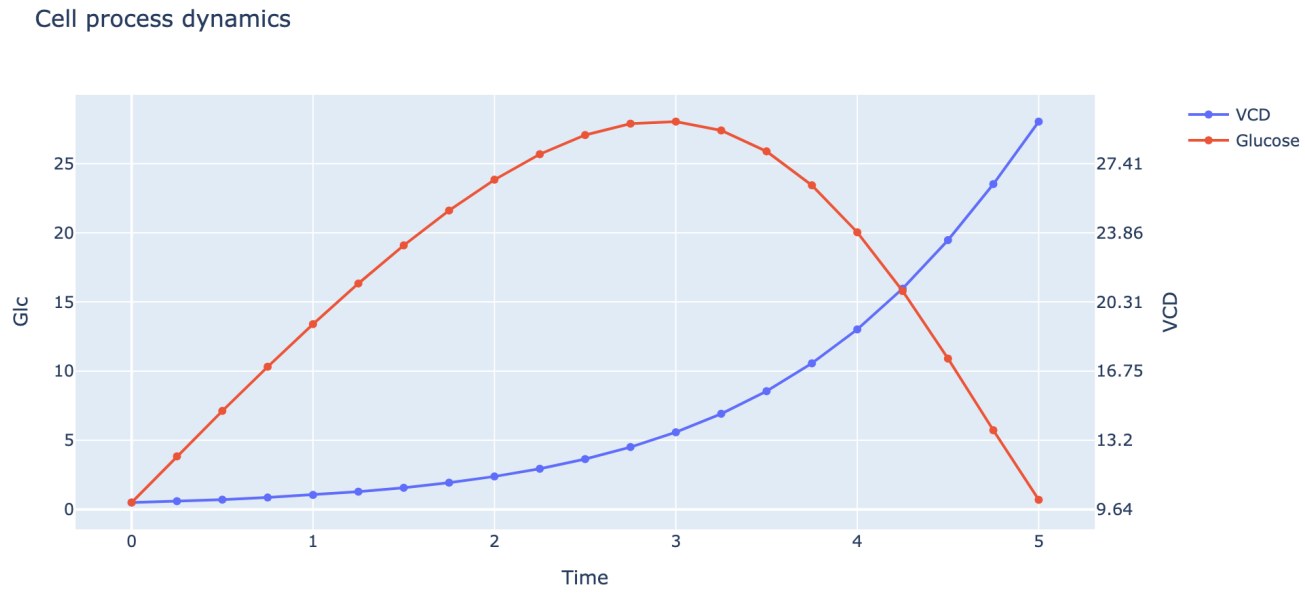
Run the base experiment

Here we work with the in silico model defined above. The process parameters that we can design are:

- `VCD_0` viable cell density at time 0
- `GLC_0` glucose concentration at time 0
- `FEED_RATE` amount of continuous glucose feed per day (feeding starts at day 0)
- `FEED_END` day at which feeding stops, which also defines the length of the experiment

Subsequently we plot the process evolution over time.

```
""" Set Process Parameters for run generation """
VCD_0 = 0.5
GLC_0 = 10
FEED_RATE = 10
FEED_END = 5
""" Measurement granularity """
TIME_STEP = 0.25
```



Create the experimental design

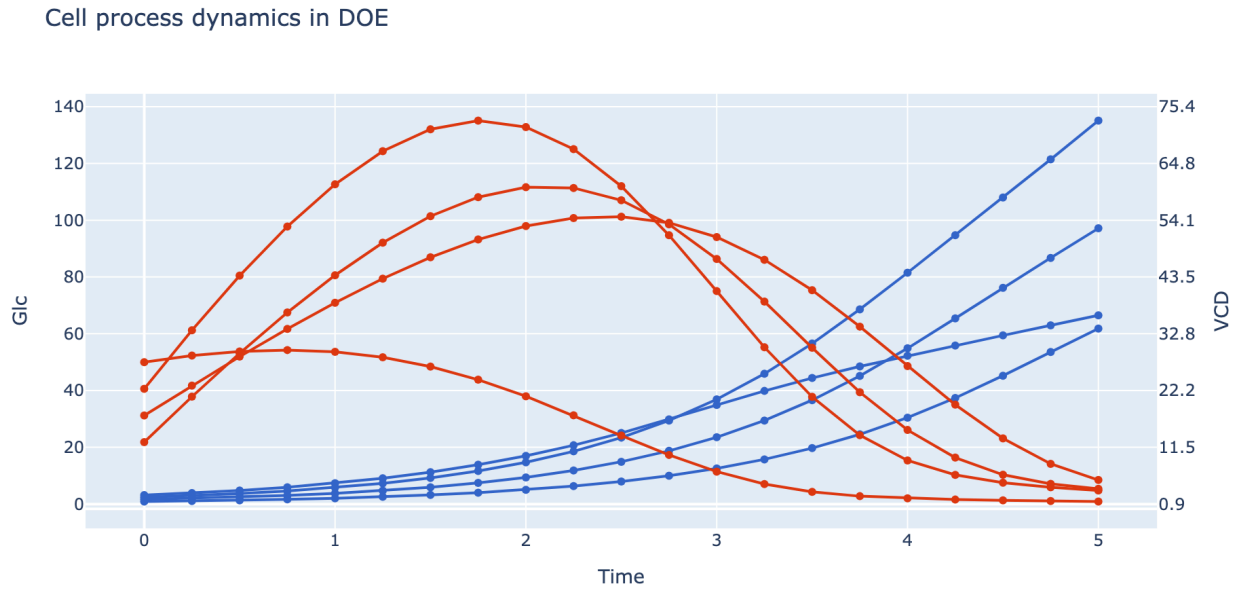
Here we create a small but diverse set of experimental conditions for the process parameters using Latin hypercube sampling and run the experiments. Each bound is given as `[lower, upper]`.

```
""" Number of experiments in training """
NUM_RUNS = 4
""" Bounds [lower, upper] of the process parameters in the DOE """
VCD_0_BOUNDS = [0.5, 3.5]
GLC_0_BOUNDS = [10, 30]
FEED_RATE_BOUNDS = [5, 55]
```

```
# Latin hypercube design scaled to the parameter bounds
def scale_doe(unit_samples, bounds_list):
    doe = np.zeros_like(unit_samples)
    for j, (lo, hi) in enumerate(bounds_list):
        doe[:, j] = lo + (hi - lo) * unit_samples[:, j]
    return doe

BOUNDS = [VCD_0_BOUNDS, GLC_0_BOUNDS, FEED_RATE_BOUNDS]
sampler = qmc.LatinHypercube(d=3, seed=42, scramble=False)
doe_train = scale_doe(sampler.random(n=NUM_RUNS), BOUNDS)
LEN_RUNS = int(np.ceil((FEED_END/TIME_STEP)+1))
# Run experiments
vcd = np.zeros((LEN_RUNS,NUM_RUNS))
glc = np.zeros((LEN_RUNS,NUM_RUNS))
for i in range(NUM_RUNS):
    t, vcd[:,i],glc[:,i] =
run_experiment(doe_train[i,0],doe_train[i,1],doe_train[i,2],FEED_END,time_step=TIME_S
TEP)
pd.DataFrame(doe_train, columns=["VCD_0", "GLC_0", "FEED_RATE"])
```

	VCD_0	GLC_0	FEED_RATE
0	3.125	27.5	11.25
1	2.375	22.5	48.75
2	1.625	12.5	36.25
3	0.875	17.5	23.75

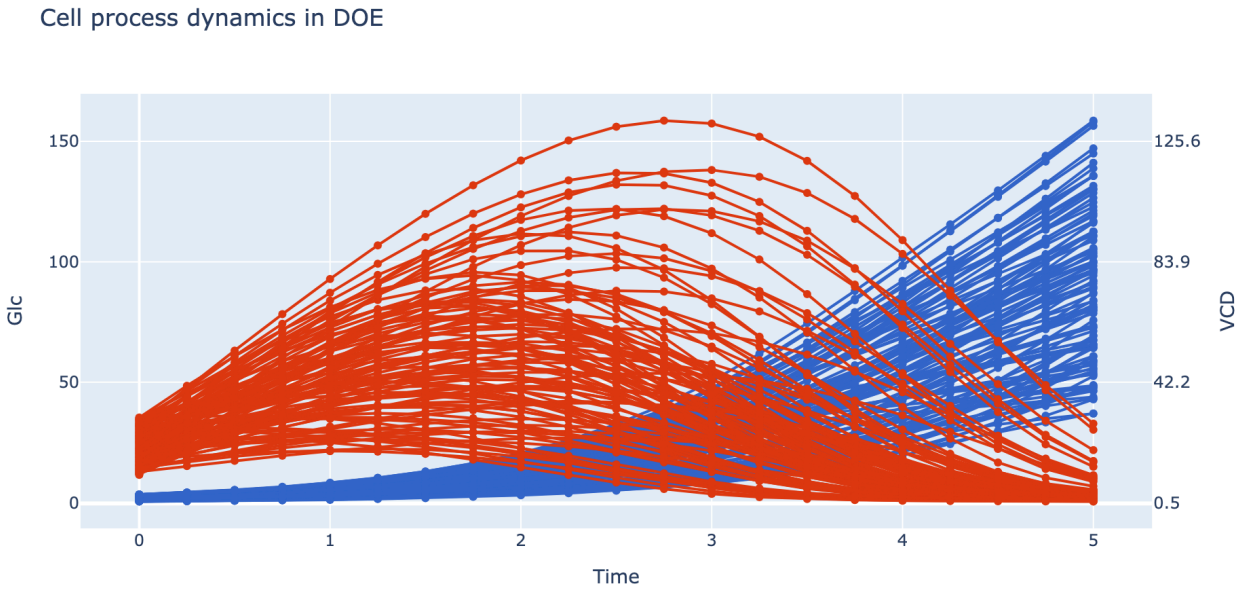


Create the test set

In the same way we create a test set with a significantly larger number of experiments to verify the performance of the models on completely unseen data.

```
""" Number of experiments in testing """
NUM_TEST = 100
```

```
# Latin hypercube design of the test set
sampler = qmc.LatinHypercube(d=3, seed=42)
doe_test = scale_doe(sampler.random(n=NUM_TEST), BOUNDS)
# Run experiments
vcd_test = np.zeros((LEN_RUNS,NUM_TEST))
glc_test = np.zeros((LEN_RUNS,NUM_TEST))
for i in range(NUM_TEST):
    t, vcd_test[:,i],glc_test[:,i] =
run_experiment(doe_test[i,0],doe_test[i,1],doe_test[i,2],FEED_END,time_step=TIME_STEP
)
```



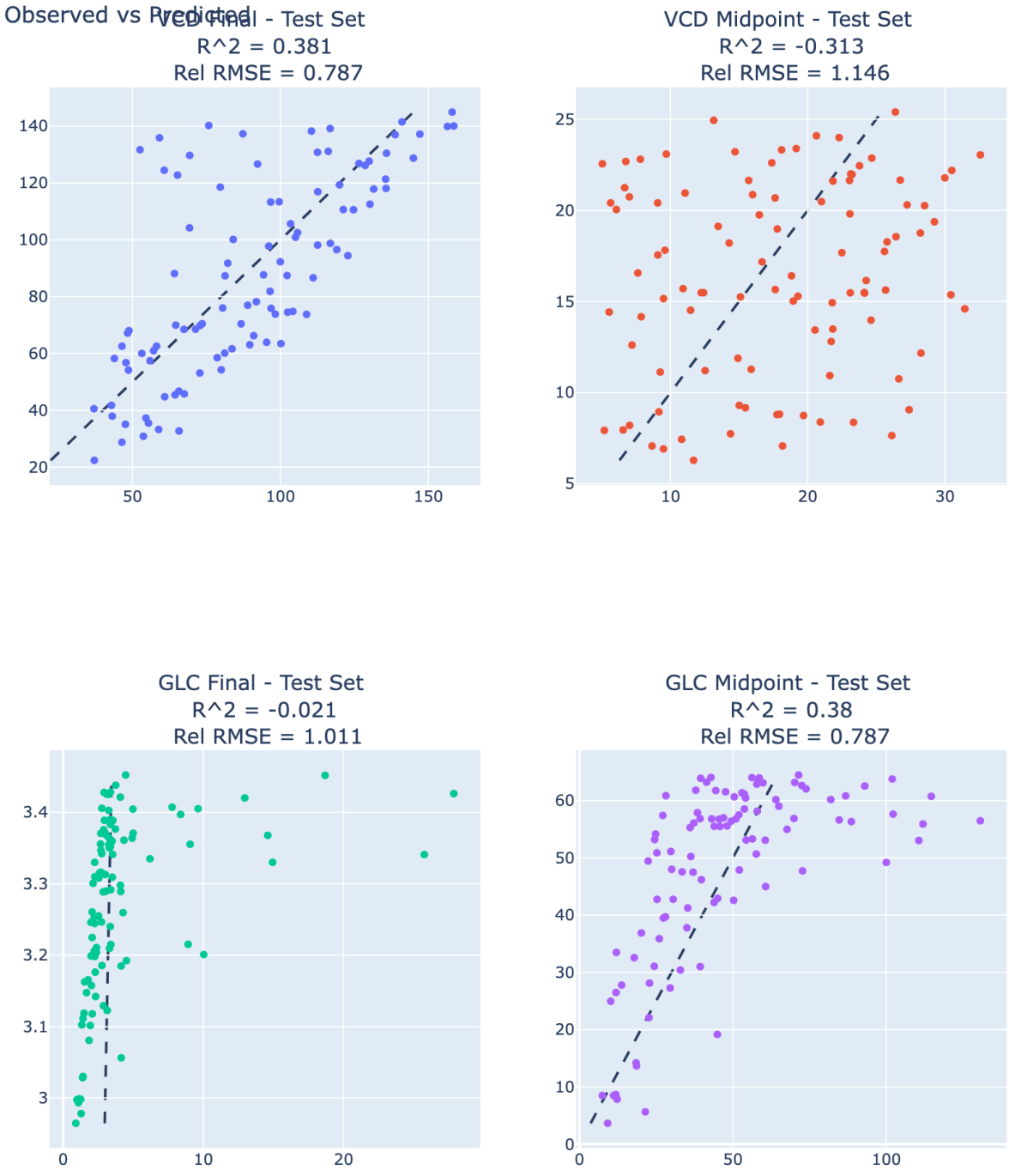
Train a black-box GP

In this section we train "black-box" Gaussian processes, which directly link the manipulated variables (the DOE) to the final VCD and the final Glc.

The scope of this exercise is to show that the good predictability of the hybrid models shown below cannot be ascribed to the fitting capabilities of GPs, or of any other model, with so few experiments.

We create models to predict VCD and Glc at the final time step as well as at the midpoint of the experiment.

```
# Define the GP kernel: scaled RBF (smooth functions) plus white noise
kernel = 1.0 * RBF(length_scale=1e-1, length_scale_bounds=(1e-2, 1e2)) +
WhiteKernel(noise_level=1e-2, noise_level_bounds=(1e-10, 1e1))
# Define Regression Model
gpr_vcd_end = GaussianProcessRegressor(kernel=kernel, random_state=0, alpha=0,
n_restarts_optimizer=3).fit(doe_train, vcd[-1,:]) # Model for final VCD
gpr_vcd_mid = GaussianProcessRegressor(kernel=kernel, random_state=0, alpha=0,
n_restarts_optimizer=3).fit(doe_train, vcd[int(LEN_RUNS/2),:]) # Model for midpoint
VCD
gpr_glc_end = GaussianProcessRegressor(kernel=kernel, random_state=0, alpha=0,
n_restarts_optimizer=3).fit(doe_train, glc[-1,:]) # Model for final Glc
gpr_glc_mid = GaussianProcessRegressor(kernel=kernel, random_state=0, alpha=0,
n_restarts_optimizer=3).fit(doe_train, glc[int(LEN_RUNS/2),:]) # Model for midpoint
Glc
# Predict values on test set
vcd_end_pred = gpr_vcd_end.predict(doe_test)
vcd_mid_pred = gpr_vcd_mid.predict(doe_test)
glc_end_pred = gpr_glc_end.predict(doe_test)
glc_mid_pred = gpr_glc_mid.predict(doe_test)
```



We can see that with only 4 training experiments the black-box models cannot learn much: the predictions on the test set are poor, in particular for the midpoint of the process.

First hybrid model

Let's suppose the following model (total rates, VCD dependent):

$$\frac{\mathrm{dVCD}}{\mathrm{dt}} = g(\mathrm{VCD}, \mathrm{Glc})$$
$$\frac{\mathrm{dGlc}}{\mathrm{dt}} = -k(\mathrm{VCD}, \mathrm{Glc}) + \mathrm{feed}$$

The functions g and k have to learn the combination of the intrinsic cell behaviour (such as the cell-specific glucose consumption) with the total amount of cells. For Glc, k thus estimates the **total** glucose consumption rate.

To obtain the full profiles in prediction, the ODE system is integrated with the learned rates, which for a single explicit Euler step reads

$$VCD_{t+\Delta t} = VCD_t + \frac{\mathrm{dVCD}}{\mathrm{dt}}\Delta t$$

Let's use the inverse method to get the model. First we estimate the derivatives at each time step with finite differences of subsequent measurements:

$$\frac{\mathrm{dVCD}}{\mathrm{dt}} \approx \frac{VCD_{t+\Delta t} - VCD_{t-\Delta t}}{2\Delta t}$$

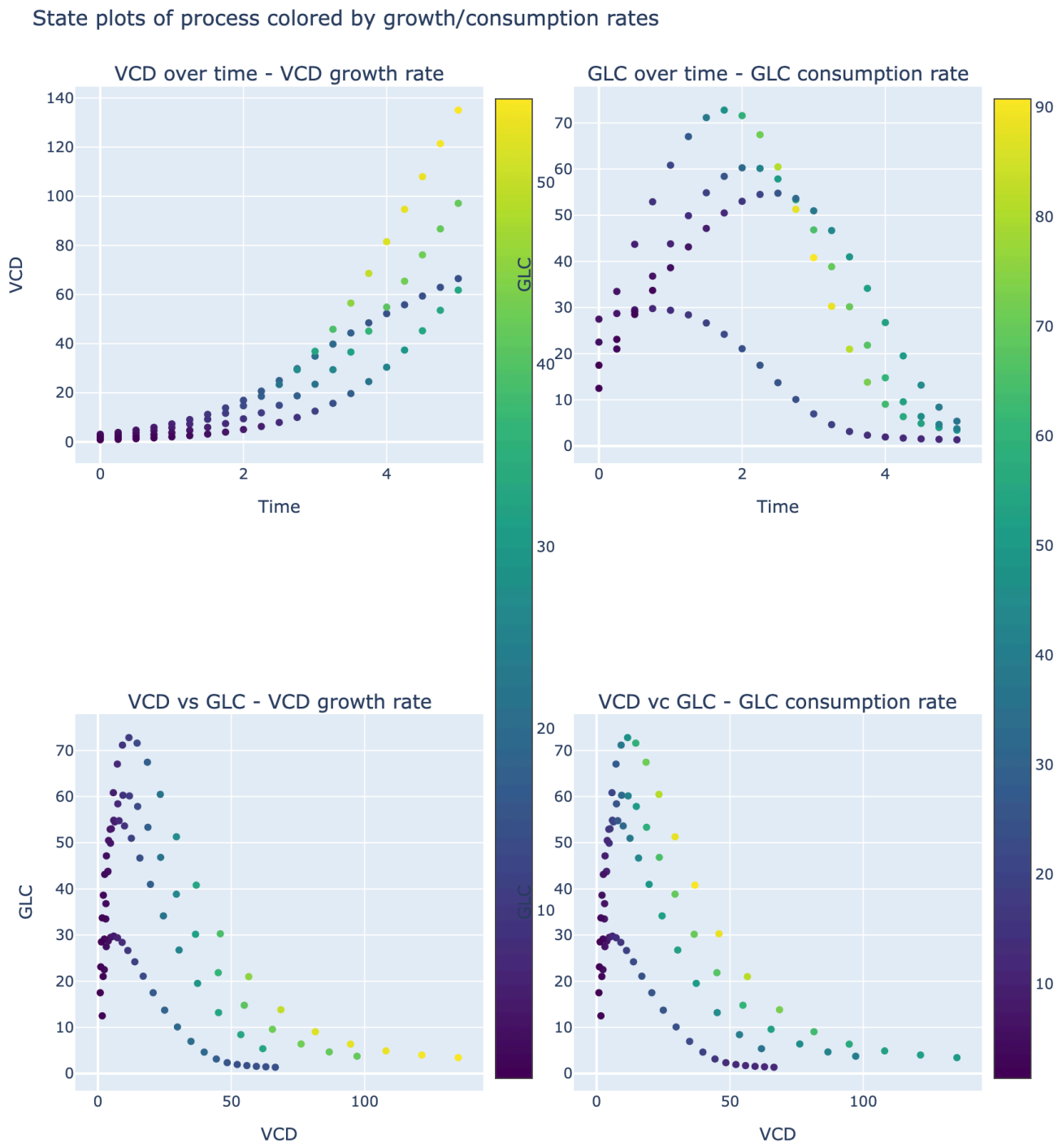
Estimate and model the derivatives

The rate g is the finite difference of VCD. The rate k is the finite difference of Glc corrected for the known feed.

```
# Estimating derivatives (changes/deltas)
g = np.zeros((LEN_RUNS, NUM_RUNS))
k = np.zeros((LEN_RUNS, NUM_RUNS))
# initial derivatives
g[0,:] = (vcd[1,:]-vcd[0,:])/TIME_STEP
dGlcdt = (glc[1,:]-glc[0,:])/TIME_STEP
k[0,:] = -dGlcdt+doe_train[:,2].T
# timestep derivatives
for j in range(1,LEN_RUNS-1):
    g[j,:] = 0.5*(vcd[j+1,:]-vcd[j-1,:])/TIME_STEP
    dGlcdt = 0.5*(glc[j+1,:]-glc[j-1,:])/TIME_STEP
    k[j,:] = -dGlcdt+doe_train[:,2].T
# final derivatives
g[-1,:] = (vcd[-1,:]-vcd[-2,:])/TIME_STEP
dGlcdt = (glc[-1,:]-glc[-2,:])/TIME_STEP
k[-1,:] = -dGlcdt+doe_train[:,2].T
```

```
# Visualize the estimated VCD growth rates (rows: time steps, columns: runs)
pd.DataFrame(g).round(3)
```

	0	1	2	3
0	2.956	2.241	1.375	0.777
1	3.316	2.574	1.599	0.886
2	4.121	3.316	2.087	1.131
3	5.108	4.236	2.676	1.437
4	6.313	5.391	3.407	1.819
5	7.768	6.845	4.325	2.301
6	9.497	8.676	5.479	2.907
7	11.503	10.974	6.928	3.670
8	13.742	13.845	8.745	4.630
9	16.080	17.402	11.010	5.832
10	18.233	21.750	13.814	7.335
11	19.732	26.949	17.246	9.202
12	20.048	32.929	21.365	11.502
13	19.022	39.343	26.150	14.301
14	17.251	45.392	31.393	17.637
15	15.674	49.920	36.540	21.476
16	14.748	52.235	40.605	25.620
17	14.351	52.987	42.604	29.570
18	14.220	53.450	42.614	32.419
19	14.192	54.176	41.946	33.203
20	14.191	54.603	41.655	32.977



Let's fit Gaussian processes for the two rates. The inputs are the states (VCD, Glc) and the outputs are the estimated derivatives:

```
X = np.stack((vcd.flatten(), glc.flatten()), axis=1)
g_mld = GaussianProcessRegressor(kernel=kernel, random_state=0, alpha=0.0,
n_restarts_optimizer=3).fit(X, g.flatten())
k_mld = GaussianProcessRegressor(kernel=kernel, random_state=0, alpha=0.0,
n_restarts_optimizer=3).fit(X, k.flatten())
```

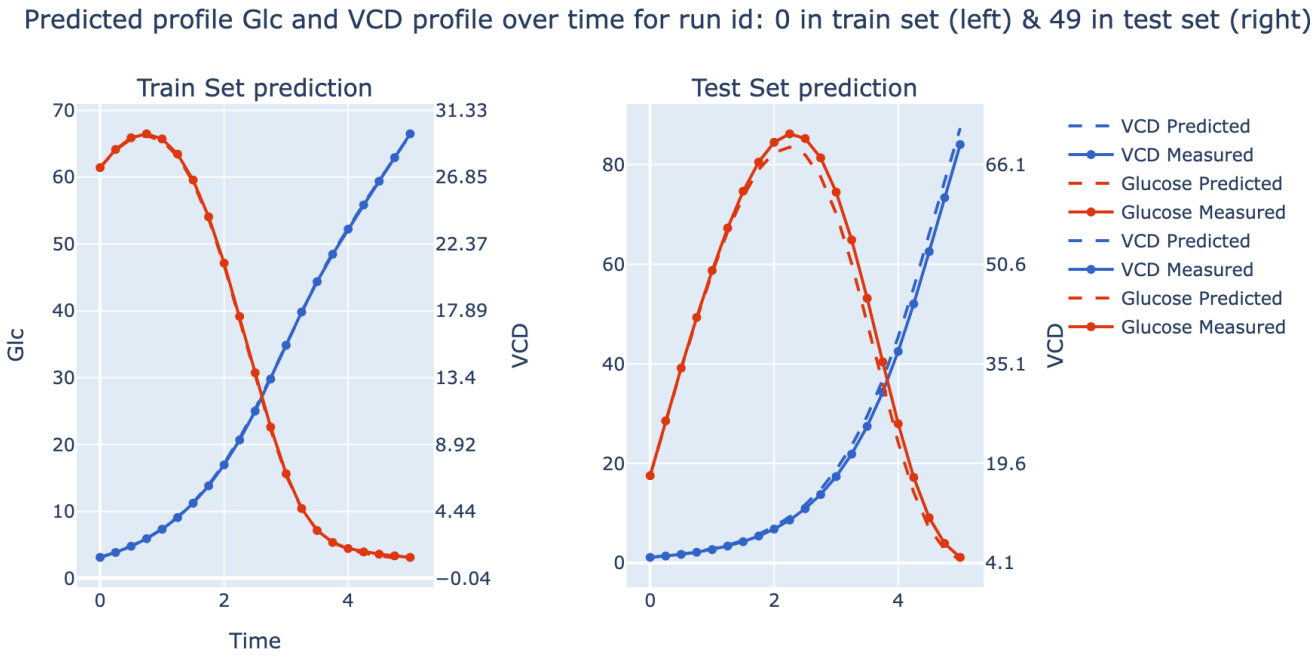
Model evaluation on the train and test set

The hybrid model is integrated over time for every experiment of the training and test set:

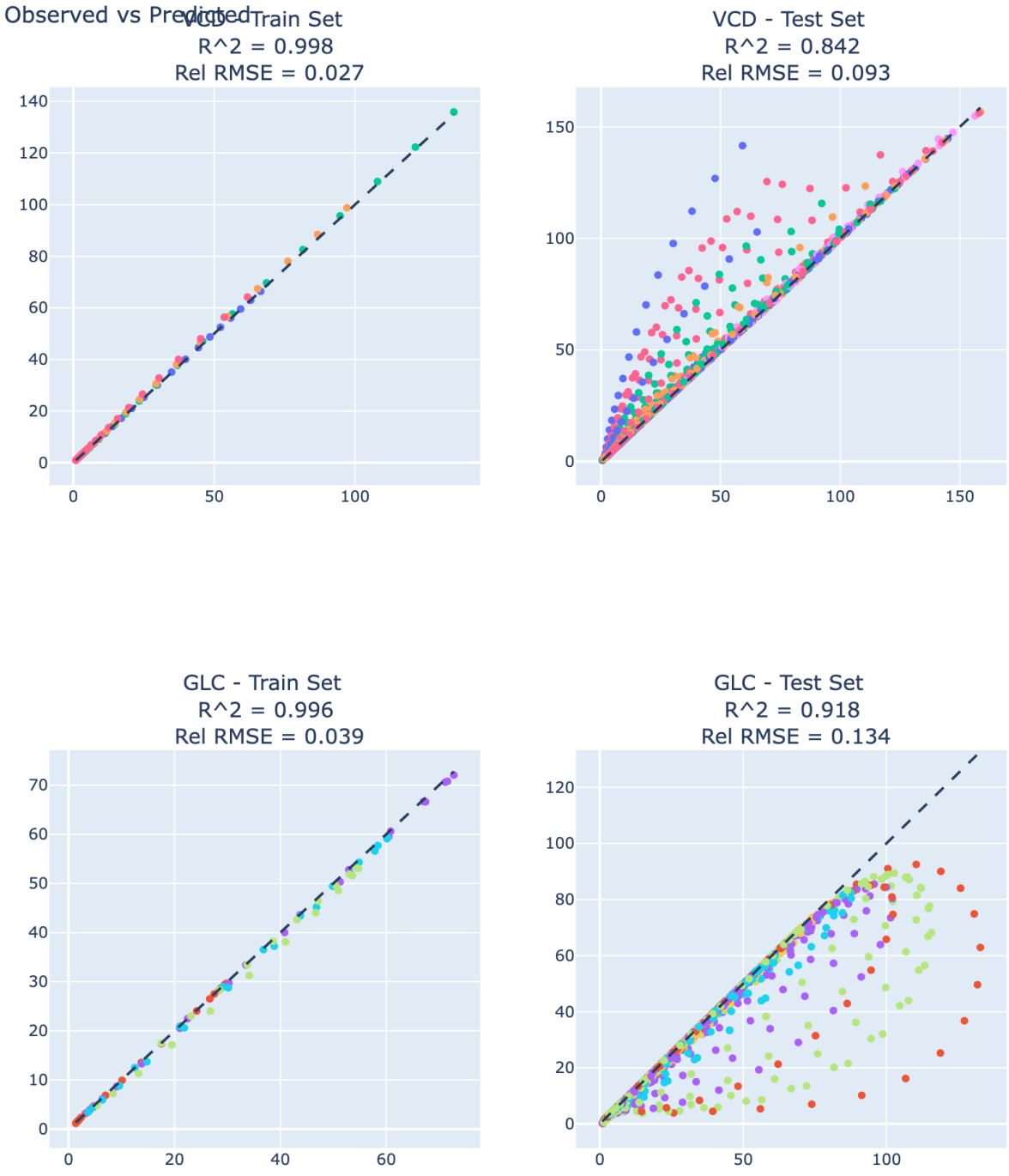
```
# Predictions in train set
vcd_train_pred = np.zeros((LEN_RUNS, NUM_RUNS))
glc_train_pred = np.zeros((LEN_RUNS, NUM_RUNS))
for i in range(NUM_RUNS):
    t,vcd_train_pred[:,i],glc_train_pred[:,i] = run_1st_hybrid(doe_train[i,0],
doe_train[i,1], doe_train[i,2], FEED_END, g_mld, k_mld, time_step=TIME_STEP)
# Predictions in test set
vcd_test_pred = np.zeros((LEN_RUNS,NUM_TEST))
glc_test_pred = np.zeros((LEN_RUNS,NUM_TEST))
for i in range(NUM_TEST):
    t, vcd_test_pred[:,i], glc_test_pred[:,i] = run_1st_hybrid(doe_test[i,0],
doe_test[i,1], doe_test[i,2], FEED_END, g_mld, k_mld, time_step=TIME_STEP)
```

Plot the fit of the first hybrid model over time for one training and one test experiment:

```
""" Select experiment on which to predict """
PLOT_TRAIN=0
PLOT_TEST=49
```



Plot observed vs predicted in the train and test set for the first hybrid model:



We can see that the model performs well on most experiments, with only 4 training runs. Compare the test set R2 with the black-box GP above.

Second hybrid model

Let's suppose the following model (cell-specific rates, VCD independent):

$$\frac{\mathrm{dVCD}}{\mathrm{dt}} = \mu(\mathrm{VCD}, \mathrm{Glc}) \cdot \mathrm{VCD}$$
$$\frac{\mathrm{dGlc}}{\mathrm{dt}} = -c(\mathrm{VCD}, \mathrm{Glc}) \cdot \mathrm{VCD} + \mathrm{feed}$$

The functions now learn the intrinsic behaviour of a single cell, such as how much glucose it consumes. These specific rates are multiplied by the total amount of cells in the process. This structure contains more mechanistic knowledge than the first hybrid model.

As before, we use the inverse method to get the model: we estimate the derivatives with finite differences and divide them by the VCD to obtain the specific rates.

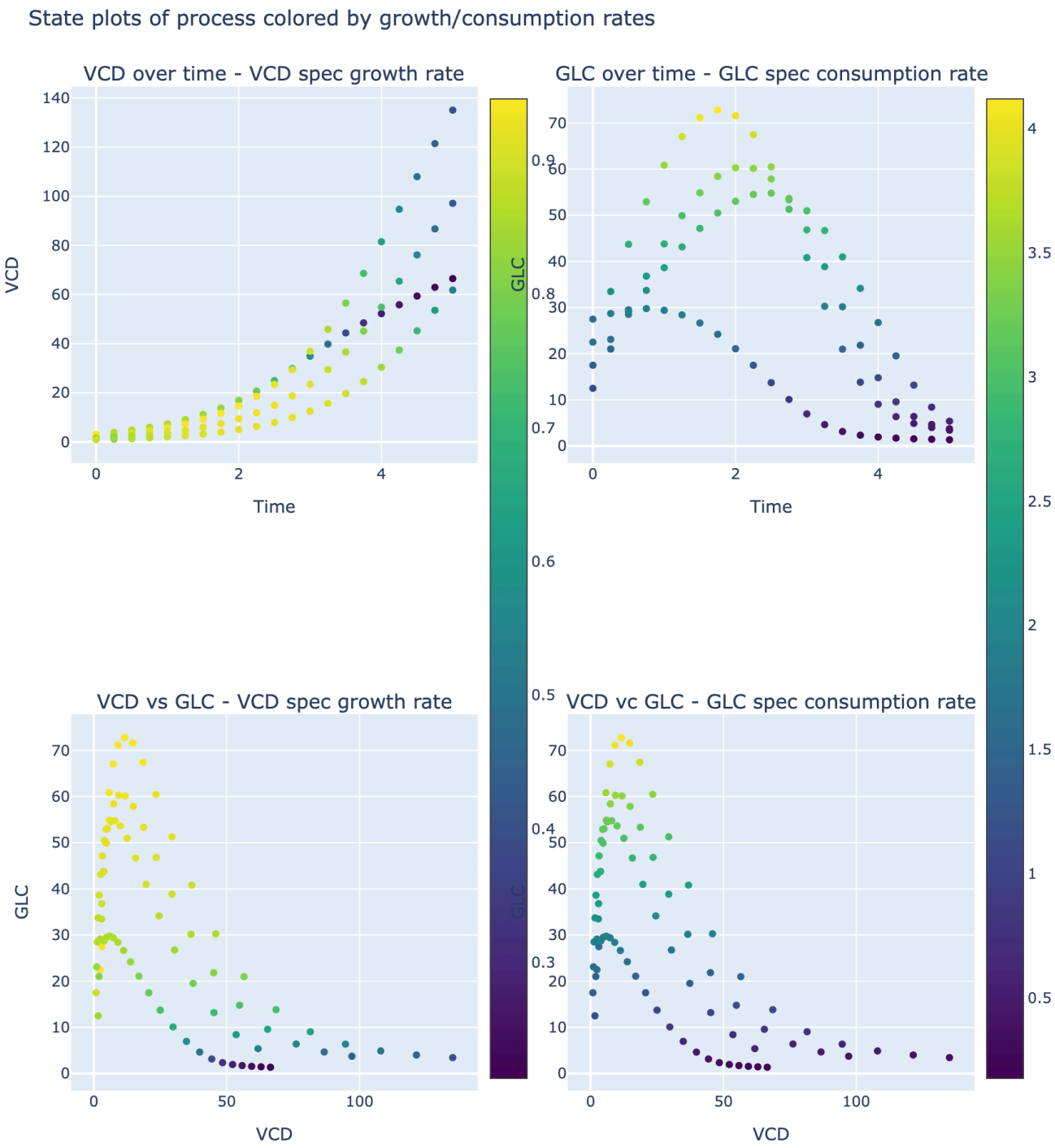
Estimate and model the specific rates

```
# Estimating derivatives (changes/deltas)
g = np.zeros((LEN_RUNS, NUM_RUNS))
k = np.zeros((LEN_RUNS, NUM_RUNS))
# initial derivatives
dVcddt = (vcd[1,:]-vcd[0,:])/TIME_STEP
g[0,:] = dVcddt/vcd[0,:]
dGlcdt = (glc[1,:]-glc[0,:])/TIME_STEP
k[0,:] = (-dGlcdt+doe_train[:,2].T)/vcd[1,:]
# timestep derivatives
for j in range(1,LEN_RUNS-1):
    dVcddt = 0.5*(vcd[j+1,:]-vcd[j-1,:])/TIME_STEP
    g[j,:] = dVcddt/vcd[j,:]
    dGlcdt = 0.5*(glc[j+1,:]-glc[j-1,:])/TIME_STEP
    k[j,:] = (-dGlcdt+doe_train[:,2].T)/vcd[j,:]
# final derivatives
dVcddt = (vcd[-1,:]-vcd[-2,:])/TIME_STEP
g[-1,:] = dVcddt/vcd[-1,:]
dGlcdt = (glc[-1,:]-glc[-2,:])/TIME_STEP
k[-1,:] = (-dGlcdt+doe_train[:,2].T)/vcd[-1,:]
```

```
# Visualize the estimated specific growth rates (rows: time steps, columns: runs)
pd.DataFrame(g).round(3)
```

	0	1	2	3
0	0.946	0.944	0.846	0.888
1	0.858	0.877	0.812	0.829
2	0.861	0.905	0.861	0.858
3	0.862	0.922	0.888	0.879
4	0.860	0.933	0.906	0.893
5	0.855	0.939	0.917	0.904
6	0.846	0.943	0.925	0.912
7	0.832	0.944	0.929	0.918
8	0.810	0.942	0.931	0.922
9	0.777	0.938	0.931	0.924
10	0.729	0.930	0.928	0.924
11	0.662	0.916	0.921	0.922
12	0.575	0.893	0.908	0.917
13	0.477	0.857	0.889	0.909
14	0.389	0.803	0.858	0.896
15	0.323	0.728	0.810	0.875
16	0.282	0.641	0.740	0.842
17	0.257	0.560	0.651	0.791
18	0.239	0.495	0.560	0.717
19	0.225	0.446	0.484	0.620
20	0.213	0.404	0.429	0.533

Let's plot the data coloured by the specific rates:



Let's fit the Gaussian processes for the specific rates:

```
X = np.stack((vcd.flatten(), glc.flatten()), axis=1)
g_mld = GaussianProcessRegressor(kernel=kernel, random_state=0, alpha=0.0,
n_restarts_optimizer=3).fit(X, g.flatten())
k_mld = GaussianProcessRegressor(kernel=kernel, random_state=0, alpha=0.0,
n_restarts_optimizer=3).fit(X, k.flatten())
```

Model evaluation on the train and test set

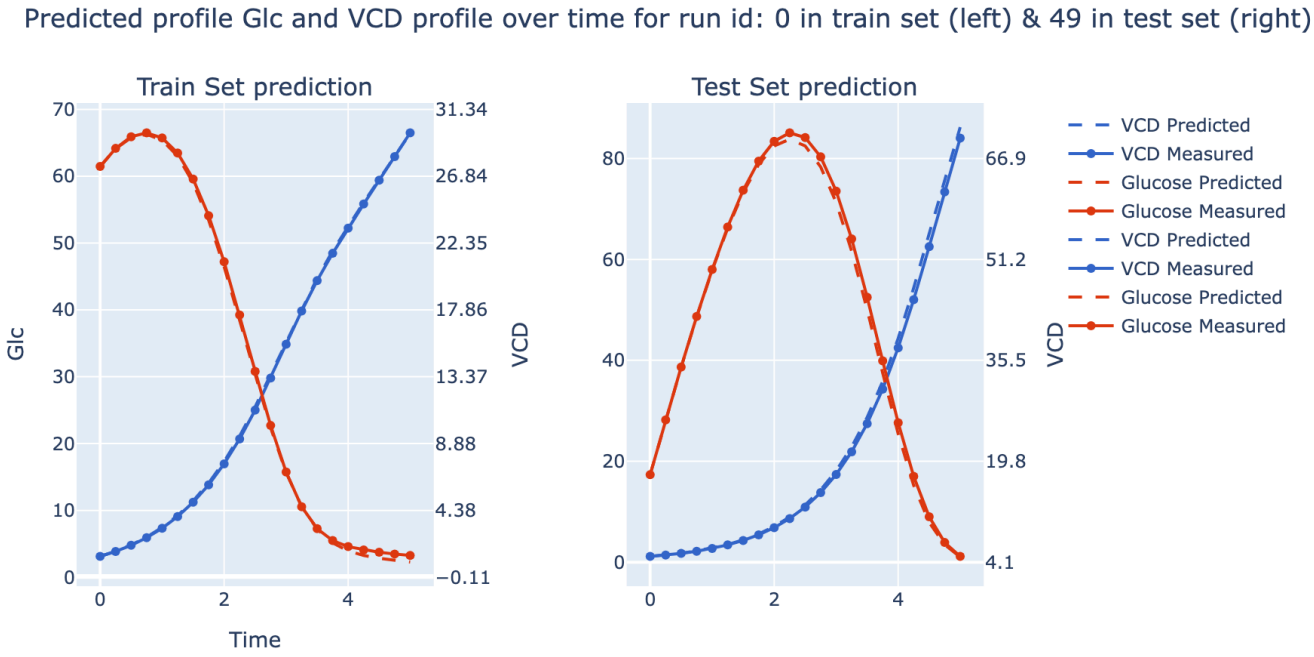
The second hybrid model is integrated over time for every experiment of the training and test set:

```
# Predictions in train set
vcd_train_pred = np.zeros((LEN_RUNS, NUM_RUNS))
glc_train_pred = np.zeros((LEN_RUNS, NUM_RUNS))
for i in range(NUM_RUNS):
    t,vcd_train_pred[:,i],glc_train_pred[:,i] = run_2nd_hybrid(doe_train[i,0],
doe_train[i,1], doe_train[i,2], FEED_END, g_mld, k_mld, time_step=TIME_STEP)
# Predictions in test set
vcd_test_pred = np.zeros((LEN_RUNS,NUM_TEST))
glc_test_pred = np.zeros((LEN_RUNS,NUM_TEST))
for i in range(NUM_TEST):
    t, vcd_test_pred[:,i], glc_test_pred[:,i] = run_2nd_hybrid(doe_test[i,0],
doe_test[i,1], doe_test[i,2], FEED_END, g_mld, k_mld, time_step=TIME_STEP)
```

Plot the fit of the second hybrid model over time for one training and one test experiment:

```
""" Select experiment on which to predict """
PLOT_TRAIN=0
PLOT_TEST=49
```

```
# Plot selected experiment run
fig = make_subplots(rows=1,cols=2,specs=[[{"secondary_y": True},{"secondary_y":
True}]],subplot_titles=("Train Set prediction","Test Set prediction"))
fig.add_trace(go.Scatter(x=t,y=vcd_train_pred[:,PLOT_TRAIN], mode='lines',name='VCD
Predicted',line=dict(color = px.colors.qualitative.G10[0], dash='dash')),row=1,col=1)
fig.add_trace(go.Scatter(x=t,y=vcd[:,PLOT_TRAIN], mode='markers+lines',name='VCD
Measured',line=dict(color = px.colors.qualitative.G10[0])),row=1,col=1)
fig.add_trace(go.Scatter(x=t,y=glc_train_pred[:,PLOT_TRAIN],
mode='lines',name='Glucose Predicted',line=dict(color = px.colors.qualitative.G10[1],
dash='dash')),secondary_y=True,row=1,col=1)
fig.add_trace(go.Scatter(x=t,y=glc[:,PLOT_TRAIN], mode='markers+lines',name='Glucose
Measured',line=dict(color =
px.colors.qualitative.G10[1])),secondary_y=True,row=1,col=1)
fig.add_trace(go.Scatter(x=t,y=vcd_test_pred[:,PLOT_TEST], mode='lines',name='VCD
Predicted',line=dict(color = px.colors.qualitative.G10[0], dash='dash')),row=1,col=2)
fig.add_trace(go.Scatter(x=t,y=vcd_test[:,PLOT_TEST], mode='markers+lines',name='VCD
Measured',line=dict(color = px.colors.qualitative.G10[0])),row=1,col=2)
fig.add_trace(go.Scatter(x=t,y=glc_test_pred[:,PLOT_TEST], mode='lines',name='Glucose
Predicted',line=dict(color = px.colors.qualitative.G10[1],
dash='dash')),secondary_y=True,row=1,col=2)
fig.add_trace(go.Scatter(x=t,y=glc_test[:,PLOT_TEST],
mode='markers+lines',name='Glucose Measured',line=dict(color =
px.colors.qualitative.G10[1])),secondary_y=True,row=1,col=2)
fig.update_layout(showlegend=True,title="Predicted profile Glc and VCD profile over
time for run id: "+str(PLOT_TRAIN)+" in train set (left) & "+str(PLOT_TEST)+" in test
set (right)",xaxis_title="Time",yaxis_title="Glc")
fig.update_yaxes(title_text="VCD", secondary_y=True)
fig.show()
```



Plot observed vs predicted in the train and test set for the second hybrid model:

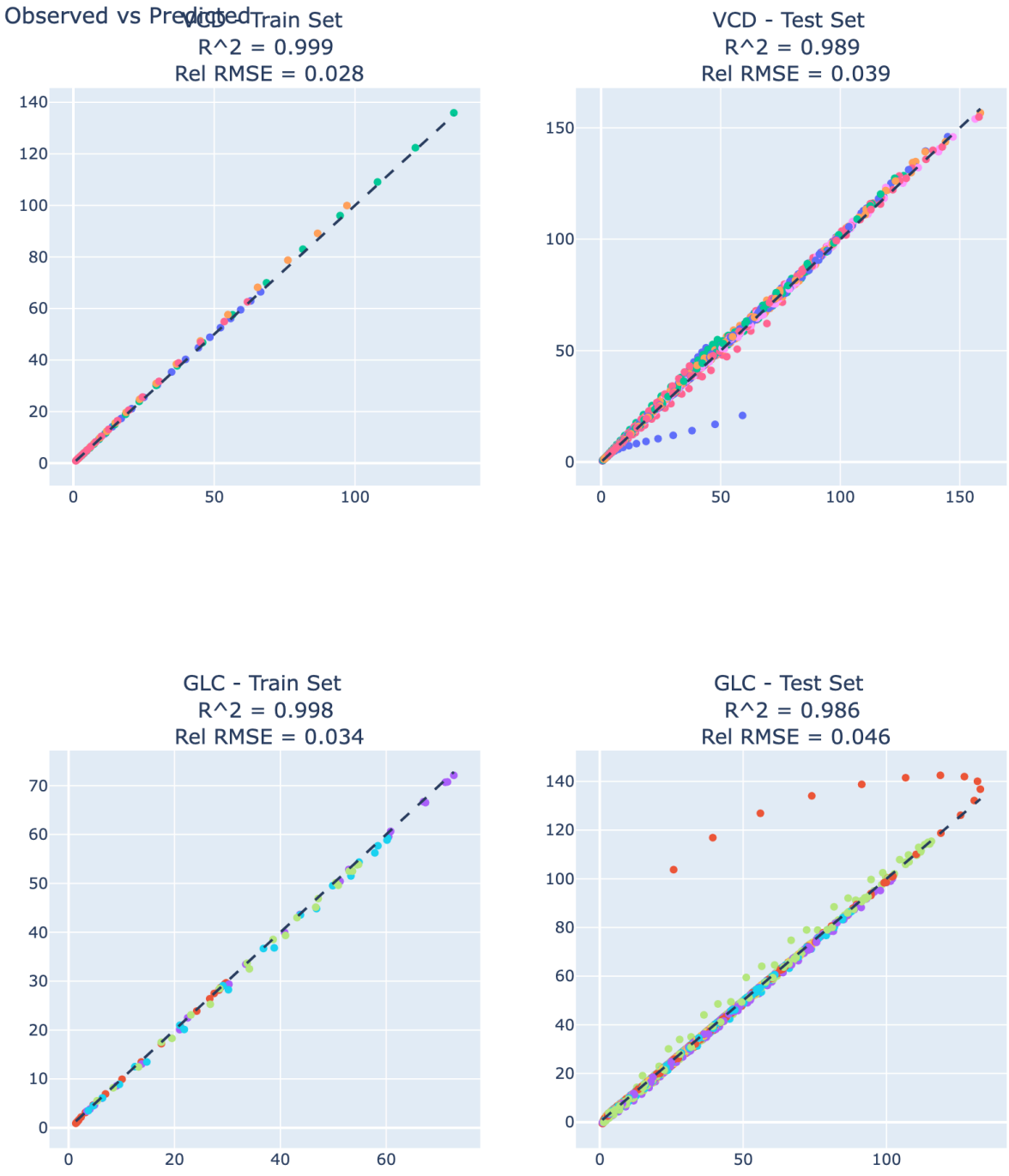
```
# Plot observed vs predicted
fig = make_subplots(rows=2, cols=2, subplot_titles=(
    f"VCD - Train Set <br> R^2 = {round(r2_score(vcd,vcd_train_pred),3)} <br> Rel
RMSE = {round(root_mean_squared_error(vcd, vcd_train_pred) /
np.std(np.array(vcd)),3)}" ,
    f"VCD - Test Set <br> R^2 = {round(r2_score(vcd_test,vcd_test_pred),3)} <br> Rel
RMSE = {round(root_mean_squared_error(vcd_test, vcd_test_pred) /
np.std(np.array(vcd_test)),3)}" ,
    f"GLC - Train Set <br> R^2 = {round(r2_score(glc,glc_train_pred),3)} <br> Rel
RMSE = {round(root_mean_squared_error(glc, glc_train_pred) /
np.std(np.array(glc)),3)}" ,
    f"GLC - Test Set <br> R^2 = {round(r2_score(glc_test,glc_test_pred),3)} <br> Rel
RMSE = {round(root_mean_squared_error(glc_test, glc_test_pred) /
np.std(np.array(glc_test)),3)}"))
for i in range(NUM_RUNS):

fig.add_trace(go.Scatter(x=vcd[:,i],y=vcd_train_pred[:,i],mode="markers",name=f"run
id {i}"),row=1,col=1)

fig.add_trace(go.Scatter(x=glc[:,i],y=glc_train_pred[:,i],mode="markers",name=f"run
id {i}"),row=2,col=1)
fig.add_shape(type="line",x0=vcd.min(),y0=vcd.min(),x1=vcd.max(),y1=vcd.max(),
layer='above', line=dict(dash='dash'),row=1,col=1)
fig.add_shape(type="line",x0=glc.min(),y0=glc.min(),x1=glc.max(),y1=glc.max(),
layer='above', line=dict(dash='dash'),row=2,col=1)
for i in range(NUM_TEST):

fig.add_trace(go.Scatter(x=vcd_test[:,i],y=vcd_test_pred[:,i],mode="markers",name=f"r
un id {i}"),row=1,col=2)

fig.add_trace(go.Scatter(x=glc_test[:,i],y=glc_test_pred[:,i],mode="markers",name=f"r
un id {i}"),row=2,col=2)
fig.add_shape(type="line",x0=vcd_test.min(),y0=vcd_test.min(),x1=vcd_test.max(),y1=vc
d_test.max(), layer='above', line=dict(dash='dash'),row=1,col=2)
fig.add_shape(type="line",x0=glc_test.min(),y0=glc_test.min(),x1=glc_test.max(),y1=gl
c_test.max(), layer='above', line=dict(dash='dash'),row=2,col=2)
fig.update_layout(title_text = "Observed vs Predicted",showlegend=False, height=1000)
fig.show()
```



Discussion: advantages and disadvantages of hybrid models

Pros:

- Work well with small datasets
- Describe the full dynamics, not only the final outcome
- Extrapolation is possible, because the mass balances hold outside the training region
- Incorporate domain knowledge
- ...

Cons:

- Prone to failing due to data mistakes (the derivatives amplify measurement noise)
- Units must be consistent
- Prone to misspecification of the process dynamics
- ...

Tasks

1. Reduce `NUM_RUNS` to 2 or increase it to 8. How does the test set performance of the black-box GP and of the hybrid models change?
2. Increase the measurement interval `TIME_STEP` to 1 day. How does the quality of the finite-difference derivatives, and with it the hybrid model, change?
3. Which of the two hybrid models would you expect to extrapolate better to experiments with a much higher initial VCD, and why?

```
# Your code here
```